AREST: Applicative REpresentational State Transfer

Composing Backus's AST with Fielding's REST
for Specification-Equivalent Application Frameworks

Samuel Lippert
Drivly Inc

Abstract

We present AREST (**A**pplicative **RE**presentational **S**tate **T**ransfer), a compositional architecture that unifies Backus's Applicative State Transition systems (1978) with Fielding's Representational State Transfer (2000). Domain knowledge is expressed as first-order logic propositions in controlled natural language (FORML 2 readings), compiled into executable functions via Backus's combining forms, and rendered as hypermedia representations with state machine transitions as navigation links. The representational state being transferred is the applicative state being transformed. We demonstrate that this composition eliminates the specification–implementation gap: the natural language reading is simultaneously the specification, the executable, and the proof obligation.

1 Introduction

Application development produces two artifacts, a specification and an implementation, that diverge over time. The specification states "each customer has exactly one email"; the implementation enforces this through database constraints, API validation, frontend checks, and tests. These are four independent projections of the same proposition that can disagree.

We observe that this divergence arises from treating specification and implementation as separate artifacts. If the sentence

Each Customer has exactly one Email.

is simultaneously a human-readable requirement and a machine-executable uniqueness–mandatory constraint, no translation step exists to introduce error.

AREST achieves this by composing two state transfer systems that operate at orthogonal layers:

Definition 1 (AST)**.** An *Applicative State Transition* system [1] consists of a set of objects O, a set of functions $F : O \to O$, and a single operation, application: $f : x$. The state $s \in O$ is transformed by applying a command: $s_{n+1} = f : s_n$. Programs are point-free compositions of combining forms. No variables are named; no state is mutated.

Definition 2 (REST)**.** A *Representational State Transfer* system (Fielding, 2000) communicates state between distributed components by transferring representations via a uniform interface. Hypermedia links in the representation encode valid state transitions (HATEOAS).

Definition 3 (AREST)**.** An *Applicative REpresentational State Transfer* system is the composition AST$\circ$REST: the representational state being transferred is the applicative state being transformed, rendered as hypermedia with transition links.

2 Theoretical Foundations

AREST draws on five theoretical traditions.

2.1 First-Order Predicate Logic

Domain knowledge in AREST is expressed as propositions in first-order logic (FOL):

$$\forall p\, \forall c_1\, \forall c_2\, (\mathrm{born}(p, c_1) \wedge \mathrm{born}(p, c_2) \rightarrow c_1 = c_2) \tag{1}$$

Equation (1) is verbalized as: "Each Person was born in at most one Country." The verbalization and the formula are isomorphic, the same proposition in two notations.

2.2 Lambda Calculus

Church's lambda calculus [2] provides the computation model. A graph schema with roles bound is a lambda term:

$$\lambda x.\ \mathrm{belongsTo}(x, \mathrm{org}) \tag{2}$$

Beta reduction (substituting a value for the bound variable) is query execution. A fully applied term (all roles bound) is a fact. A partially applied term is a query.

2.3 FP Algebra and AST

Backus [1] introduced function-level programming: programs built from combining forms without naming variables.

Definition 4 (Combining Forms)**.** Let $f, g, f_1, \ldots, f_n$ be functions and x an object.

$$[f_1, \ldots, f_n] : x = \langle f_1 : x, \ldots, f_n : x \rangle \qquad \text{(Construction)} \tag{3}$$

$$(f \circ g) : x = f : (g : x) \qquad \text{(Composition)} \tag{4}$$

$$(\alpha f) : \langle x_1, \ldots, x_n \rangle = \langle f : x_1, \ldots, f : x_n \rangle \qquad \text{(Apply-to-All)} \tag{5}$$

$$(/f) : \langle x_1, \ldots, x_n \rangle = f : \langle x_1, /f : \langle x_2, \ldots \rangle \rangle \qquad \text{(Insert)} \tag{6}$$

$$(p \rightarrow f; g) : x = \begin{cases} f : x & \text{if } p : x = T \\ g : x & \text{if } p : x = F \end{cases} \qquad \text{(Condition)} \tag{7}$$

$$(\mathrm{bu}\ f\ a) : x = f : \langle a, x \rangle \qquad \text{(Binary-to-Unary)} \tag{8}$$

In AREST, these forms have precise domain interpretations:

Backus's AST extends FP with explicit state: the system state is an object s, and a program p transforms it: $s' = p : s$. The system evolves by successive application. In AREST, the state is the population of facts, and each API request is a command applied to the population.

Backus Form	AREST Interpretation
Construction $[f_1, \ldots, f_n]$	Graph Schema (CONS of Roles)
Selector s	Role at position s
Composition $f \circ g$	Derivation rule chain
Apply-to-All αf	Population traversal
Insert $/f$	Aggregation (exists, forall)
Condition $p \to f; g$	Constraint evaluation, guards
Binary-to-Unary bu $f\ x$	Partial application (query)
Constant $\bar{x}$	Literal value in a reading

2.4 Object-Role Modeling

ORM2 [5] models information as elementary facts, where objects play roles in predicates. The surface syntax, FORML 2 [7], is a controlled natural language:

```
Organization(.Slug) is an entity type.
Organization has Name.
  Each Organization has exactly one Name.
```

Each sentence is a FOL proposition. The reference scheme `(.Slug)` defines the entity's identity predicate. The constraint `Each Organization has exactly one Name` combines a uniqueness constraint (UC, "at most one") with a mandatory constraint (MC, "at least one"). The CSDP (Conceptual Schema Design Procedure) [5] provides a 7-step procedure guaranteeing semantic completeness, and the RMAP (Relational Mapping Procedure) [5] maps conceptual schemas to relational schemas.

The key property: the verbalization is lossless. Every ORM2 constraint has exactly one canonical verbalization pattern, and every well-formed verbalization maps to exactly one constraint. The grammar is restrictive enough to parse deterministically.

ORM2 models state machines as facts about facts. A State Machine instance is a separate entity from the Resource it governs, connected by the fact type `State Machine is for Resource` (1:1). Objectification of such fact types requires a spanning uniqueness constraint [6]. A state transition creates a ternary fact `Event triggered Transition in State Machine`, forming the complete audit trail as facts in the population.

2.5 The Relational Model

Codd's relational model [3] provides the storage semantics. RMAP [5] maps ORM2 conceptual schemas to Codd's relations following formal rules: uniqueness constraints become keys, mandatory constraints become NOT NULL, subset constraints become foreign keys, and exclusive-or constraints become CHECK constraints.

In AREST, each entity instance is an independent storage unit keyed by its reference scheme. The entity data contains the dependent attributes. Each unit satisfies Codd's third normal form: every attribute depends on the key, the whole key, and nothing but the key. The State Machine is a separate entity with its own key, not a denormalized field on the Resource.

2.6 REST

Fielding's architectural style [4] constrains distributed systems to transfer representations of state via a uniform interface. The critical constraint:

Hypermedia as the engine of application state (HATEOAS): each response includes links encoding valid state transitions.

In AREST, the state machine's valid transitions from the current status serve as the HATEOAS links. The API response for an entity in status "Draft" includes actions `[place, cancel]`, which are transitions compiled from the readings.

3 The AREST Execution Model

3.1 Commands as Function Application

Every API interaction is a Command: a function from population to population.

$$\text{Command} : \text{Population} \to (\text{Population}', \text{Representation}) \tag{9}$$

A CreateEntity command is a composed function that extends the population with new formulae:

$$\text{create} = \text{emit} \circ \text{derive} \circ \text{validate} \circ \text{resolve} \tag{10}$$

where:

- resolve: applies the reference scheme selector to determine the entity's identity
- validate: evaluates constraints against the current population
- derive: forward-chains derivation rules to fixed point, adding derived formulae (including state machine initialization) to the population
- emit: produces the new population containing the entity, its state machine instance, and all derived facts

State machine initialization is not a separate step. The state machine definition is a set of formulae in the population. When the entity is created, the derivation rules produce the State Machine instance and its initial Status as derived facts. The composition is compiled from the readings. The engine applies it as one reduction.

3.2 State Machines as Fold

A state machine is Backus's Insert (fold) over an event stream:

$$\text{run_machine} = /\text{transition} : \langle e_1, \ldots, e_n \rangle \tag{11}$$

Each transition is a Condition:

$$\text{transition} = (p \to \bar{s}'; \text{id}) \tag{12}$$

where $p = \text{eq} \circ [\text{id}, \overline{\langle s_{\text{from}}, e \rangle}]$ checks whether the current state and event match, and $\bar{s}'$ is the constant function returning the target state.

The transition table compiles to nested conditions. The first matching condition fires; the rest fall through to identity (state unchanged):

$$f = (p_1 \to \bar{s_1};\ (p_2 \to \bar{s_2};\ \ldots;\ \mathrm{id})) \tag{13}$$

Guards are constraints that must produce zero violations for the transition to fire:

$$\text{guarded_transition} = (\text{guard_passes} \wedge p \to \bar{s'};\ \mathrm{id}) \tag{14}$$

where guard_passes = null $\circ$ evaluate_constraints.

3.3 Derivation Rules as Policy

Any policy that can be expressed as a derivation rule is evaluated by the engine. Access control, visibility scoping, and domain evolution are all derivation rules compiled to Composition chains. For example, the access rule

```
User can access Domain iff User has Org Role
  in Organization and Domain belongs to
  that Organization.
```

compiles to:

$$\text{access} = /\text{or} \circ \alpha(\text{bu eq } d_{\text{org}}) : \text{user_orgs} \tag{15}$$

Forward chaining from the user's identity resolves the accessible domains. The derivation rule is the policy.

Domain evolution follows the same pattern. A proposed change to the readings is itself a fact in the population. The Curry-Howard correspondence [8] applies: proposing a new fact type is proposing a theorem, and the change is only accepted if the engine can verify consistency (the proof obligation). CSDP validation serves as the proof checker, and a successful validation is the proof.

4 Properties

Theorem 1 (Specification Equivalence)**.** In AREST, the FORML 2 reading is isomorphic to the executable constraint. No translation step exists between specification and implementation.

Proof. Each FORML 2 sentence has exactly one canonical verbalization pattern mapping to exactly one FOL formula [7]. The parser produces a schema of propositions. The engine compiles them to combining forms. Evaluation is beta reduction. The transformation is deterministic and invertible: given the compiled form, the original reading can be recovered by verbalization. □

Theorem 2 (Completeness of State Transfer)**.** A CreateEntity command produces the complete new population in one function application: the resource entity, the state machine instance, derived facts, and constraint violations.

Proof. The command is a composition of functions. The composition is applied atomically; no intermediate state is observable. The result contains the complete new population. □

Corollary 1 (HATEOAS from Readings)**.** The hypermedia links in the REST representation are projections of the compiled state machine's transition table. No URL patterns are hardcoded.

5 Implementation

The reference implementation uses a Rust/WASM functional logic engine:

- **Engine** (Rust/WASM): AST node types, compilation, evaluation, forward chaining, state machines, RMAP, CSDP validation, conceptual queries, constraint induction.
- **Storage**: one addressable unit per entity instance. Each unit is a row in the ORM population, keyed by its reference scheme.
- **Transport**: thin HTTP dispatch. Parses request, calls engine, persists result, returns representation.

The engine exposes a single entry point:

```
pub fn apply_command(
    model: &CompiledModel,
    command: &Command,
    population: &Population,
) -> CommandResult
```

The entire system reduces to one function, one application, and one state transfer.

6 Related Work

GraphQL provides a query language but does not close the specification–implementation gap. **OData** standardizes REST conventions but remains procedural. **RDF/Linked Data** represents knowledge as triples but lacks the constraint taxonomy, state machines, and algebraic computation model. **Proof-Carrying Code** [9] verifies code against safety policies but operates at the machine code level, not at the domain logic level.

AREST composes Backus's AST with Fielding's REST, two state transfer systems published 23 years apart in adjacent communities (programming languages and distributed systems) that operate at orthogonal layers of the same architecture.

7 Conclusion

AREST composes Applicative State Transition systems with Representational State Transfer. The readings are axioms. The engine applies functions to populations. The API renders the result as hypermedia. The client navigates by following links. Each link is a function application waiting to happen.

The theoretical foundations: first-order logic, Church's computation [2], Codd's relations [3], Backus's algebra [1], Halpin's modeling [5], Fielding's distribution [4]. AREST is their synthesis.

Acknowledgments

Claude (Anthropic) was used as a programming assistant during implementation of the reference system and as a drafting aid during preparation of this manuscript. All architectural decisions, theoretical contributions, and the composition of AST with REST were made by the human author.

References

[1] J. Backus, "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs," *Communications of the ACM*, vol. 21, no. 8, pp. 613–641, 1978.

[2] A. Church, "An Unsolvable Problem of Elementary Number Theory," *American Journal of Mathematics*, vol. 58, no. 2, pp. 345–363, 1936.

[3] E.F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[4] R.T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000.

[5] T. Halpin, *Information Modeling and Relational Databases*, 2nd ed. Morgan Kaufmann, 2008.

[6] T. Halpin, "Objectification and Atomicity," unpublished note, updated 2020.

[7] T. Halpin and M. Curland, "Automated Verbalization for ORM 2," in *OTM Workshops*, pp. 1181–1190, 2006.

[8] W.A. Howard, "The Formulae-as-Types Notion of Construction," in *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, pp. 479–491, 1980.

[9] G.C. Necula, "Proof-Carrying Code," *Proc. 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pp. 106–119, 1997.